Lab 25 — Data in Transit: TLS Hardening and Cipher Audit

> **SY0-701 Objective 3.3** — Compare and contrast concepts and strategies to protect data
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Stand up a deliberately weak HTTPS server, prove it accepts deprecated TLS 1.0/1.1 and non-forward-secret ciphers, harden it to TLS 1.2/1.3 with AEAD ciphers and HSTS, and confirm the improvement with OpenSSL and testssl.sh.

You'll build: An nginx HTTPS site with a hardened TLS snippet, plus before-and-after audit reports.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 07 (PKI) is useful background; tools installed in step 1.

Background

- **Data in transit** is protected by TLS for web, mail and APIs, and by IPSec or VPN tunnels at the network layer. Disk encryption does nothing for data on the wire.
- A cipher suite such as `ECDHE-RSA-AES256-GCM-SHA384` names the **key exchange** (ECDHE), **authentication** (RSA), **bulk cipher** (AES-256-GCM) and **hash** (SHA-384).
- **Forward secrecy** (ephemeral ECDHE) means a stolen server key cannot decrypt recorded past sessions. **AEAD** ciphers (AES-GCM, ChaCha20-Poly1305) give confidentiality and integrity together.
- TLS 1.0 and 1.1 are deprecated (RFC 8996). Leaving them enabled invites **downgrade attacks**; the hardened baseline is TLS 1.2 and 1.3 only.
- **HSTS** (`Strict-Transport-Security`) tells browsers to use HTTPS only, defeating SSL-stripping after the first visit.

Steps

1. Install nginx, OpenSSL and testssl.sh

Port 80 is freed first in case Apache from Lab 21 is running. testssl.sh is cloned from its official GitHub repository (3.2 stable branch).

```
systemctl stop apache2 2>/dev/null || true
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y nginx openssl git
curl bsdextrautils dnsutils
[ -d /opt/testssl ] || git clone --depth 1 --branch 3.2
https://github.com/testssl/testssl.sh.git /opt/testssl
/opt/testssl/testssl.sh --version 2>/dev/null | grep -m1 -i version || true
```

2. Create a certificate and a deliberately weak HTTPS site

The `@SECLEVEL=0` cipher string re-enables legacy protocols and ciphers, imitating an old, badly configured server.

```
openssl req -x509 -newkey rsa:2048 -nodes -days 30 \
  -keyout /etc/ssl/private/lab25.key -out /etc/ssl/certs/lab25.crt \
  -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
cat > /etc/nginx/sites-available/default <<'EOF'
server {
    listen 443 ssl default_server;
    server_name localhost;
    ssl_certificate     /etc/ssl/certs/lab25.crt;
    ssl_certificate_key /etc/ssl/private/lab25.key;
    # WEAK BASELINE - legacy protocols and every cipher
    ssl_protocols TLSv1 TLSv1.1 TLSv1.2 TLSv1.3;
    ssl_ciphers "ALL:@SECLEVEL=0";
    root /var/www/html;
}
EOF
echo "Lab 25 TLS test page" > /var/www/html/index.html
nginx -t && systemctl restart nginx
curl -skI https://127.0.0.1/ | head -1
```

3. Probe the weak site with openssl s_client

Ask for TLS 1.1 and for a non-forward-secret CBC cipher — the weak server accepts both.

```
echo | openssl s_client -connect 127.0.0.1:443 -tls1_1 -cipher 'DEFAULT:@SECLEVEL=0'
2>/dev/null | grep -E '^New,|Protocol  :'
echo | openssl s_client -connect 127.0.0.1:443 -tls1_2 -cipher 'AES128-SHA:@SECLEVEL=0'
2>/dev/null | grep -E '^New,'
```

```
New, TLSv1.0, Cipher is ECDHE-RSA-AES256-SHA
    Protocol  : TLSv1.1
New, SSLv3, Cipher is AES128-SHA
```

The `Protocol` line is the negotiated version (TLS 1.1 — deprecated). The label after `New,` is only the version that first defined the cipher; what matters on the second line is that `AES128-SHA` — static RSA key exchange, CBC mode, no forward secrecy — is accepted.

4. Run a baseline testssl.sh audit

Scan protocols, forward secrecy and headers. This takes one to two minutes.

```
/opt/testssl/testssl.sh --quiet --color 0 --warnings batch -p -f -h https://127.0.0.1 >
/root/lab25-baseline.txt 2>&1
grep -E '^ (SSLv2|SSLv3|TLS 1 |TLS 1.1|TLS 1.2|TLS 1.3)' /root/lab25-baseline.txt
grep -iE 'Strict Transport Security' /root/lab25-baseline.txt
```

Expect `TLS 1` and `TLS 1.1` marked `offered (deprecated)` and `Strict Transport Security not offered`.

5. Write a hardened TLS configuration

Allow TLS 1.2 and 1.3 only, ECDHE key exchange with AEAD ciphers, no session tickets, and HSTS for one year.

```
cat > /etc/nginx/snippets/tls-hardening.conf <<'EOF'
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-
CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-
AES128-GCM-SHA256;
ssl_prefer_server_ciphers off;
ssl_session_tickets off;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
EOF
sed -i -e '/WEAK BASELINE/d' -e '/ssl_protocols/d' -e '/ssl_ciphers/d' /etc/nginx/sites-
available/default
sed -i '/ssl_certificate_key/a\    include /etc/nginx/snippets/tls-hardening.conf;'
/etc/nginx/sites-available/default
cat /etc/nginx/sites-available/default
```

6. Reload nginx and re-test with openssl s_client

TLS 1.1 is now refused, TLS 1.3 negotiates an AEAD suite, and the HSTS header is present.

```
nginx -t && systemctl reload nginx
echo | openssl s_client -connect 127.0.0.1:443 -tls1_1 -cipher 'DEFAULT:@SECLEVEL=0' 2>&1
| grep -E '^New,|alert|no protocols' | head -2
echo | openssl s_client -connect 127.0.0.1:443 -tls1_2 -cipher 'AES128-SHA:@SECLEVEL=0'
2>&1 | grep -E '^New,|alert' | head -1
echo | openssl s_client -connect 127.0.0.1:443 -tls1_3 2>/dev/null | grep -E '^New,'
curl -skI https://127.0.0.1/ | grep -i strict-transport
```

```
New, (NONE), Cipher is (NONE)
...:tlsv1 alert protocol version:...
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
strict-transport-security: max-age=31536000; includeSubDomains
```

7. Re-audit with testssl.sh and compare

```
/opt/testssl/testssl.sh --quiet --color 0 --warnings batch -p -f -h https://127.0.0.1 >
/root/lab25-hardened.txt 2>&1
for f in baseline hardened; do
  echo "== $f"; grep -E '^ (TLS 1 |TLS 1.1|TLS 1.3)' /root/lab25-$f.txt; grep -iE 'Strict
Transport Security' /root/lab25-$f.txt
done
```

Test it

```
echo | openssl s_client -connect 127.0.0.1:443 -tls1_1 -cipher 'DEFAULT:@SECLEVEL=0' 2>&1
| grep -c 'Cipher is (NONE)'
echo | openssl s_client -connect 127.0.0.1:443 -tls1_3 2>/dev/null | grep -o 'TLSv1.3,
Cipher is [A-Z0-9_]*'
curl -skI https://127.0.0.1/ | grep -ci strict-transport-security
```

Expected: `1` (TLS 1.1 handshake fails), `TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384` (or another TLS 1.3 AEAD suite), and `1` (HSTS present). The hardened testssl report shows `TLS 1` and `TLS 1.1` as `not offered`.

Troubleshooting

Symptom	Fix
`bind() to 0.0.0.0:80 failed (98: Address already in use)` during install	Another web server holds port 80; run `systemctl stop apache2` then `apt-get install -f -y`.
`nginx: [emerg] unknown directive "include..."`	The `sed` append in step 5 lost its leading backslash; edit the site file and put the `include` line inside the `server { }` block.
testssl.sh reports `doesn't seem to be a TLS/SSL enabled server`	nginx is not listening on 443; check `nginx -t` and `ss -tlnp | grep 443`.

Challenge

Add OCSP stapling and a certificate signed by your own CA from Lab 07 instead of the self-signed one, then use `openssl s_client -status -CAfile` to verify the chain. Explain why HSTS preload must be used with care.

Reflection

A partner insists on connecting with TLS 1.0. Weigh availability against confidentiality and integrity, and name the compensating controls you would demand if a temporary exception were granted.

Exam tips

- TLS protects data in transit; FDE protects data at rest — know which control fits which data state.
- Forward secrecy comes from ephemeral (EC)DHE key exchange; static RSA key exchange has none.
- TLS 1.0/1.1 and SSL are deprecated; downgrade attacks such as POODLE abuse old versions.
- HSTS forces HTTPS in the browser and defeats SSL-stripping (an on-path attack).

References

- Mozilla SSL Configuration Generator — https://ssl-config.mozilla.org/
- RFC 8446 TLS 1.3 — https://www.rfc-editor.org/rfc/rfc8446
- RFC 8996 Deprecating TLS 1.0 and TLS 1.1 — https://www.rfc-editor.org/rfc/rfc8996
- NIST SP 800-52 Rev. 2 TLS Implementation Guidelines — https://csrc.nist.gov/pubs/sp/800/52/r2/final
- testssl.sh — https://testssl.sh/
